

Figure 12-3: The 15-puzzle has undirected edges (drawn without an arrowhead) between its nodes because slides can be undone by performing the opposite slide.

To optimize our algorithm, we'll avoid slides that undo the previous slide. However, this optimization alone won't save the algorithm from a stack overflow. While it makes the *edges* in the tree graph directed, it doesn't turn the puzzle-solver algorithm into a DAG, because it has cycles, or loops, from lower nodes to higher ones. These loops happen if you slide the tiles in a circular pattern, as in [Figure 12-4](#).

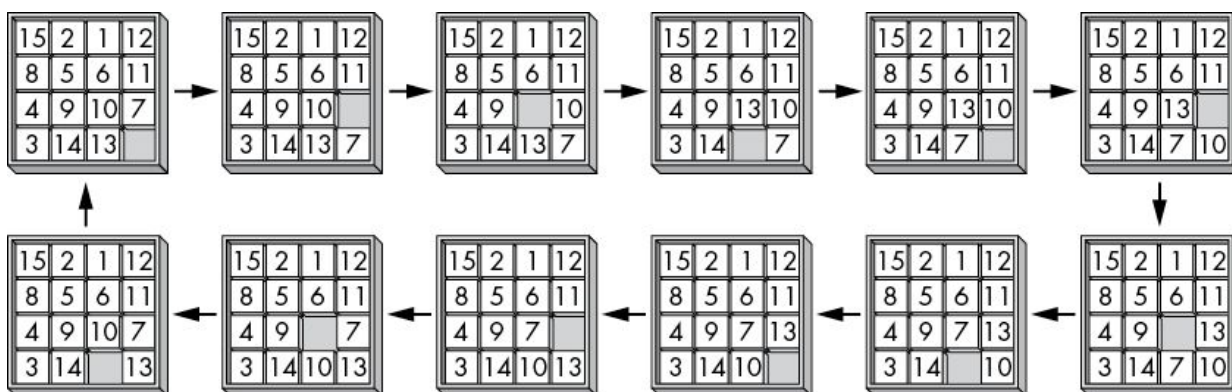


Figure 12-4: An example of a loop in the 15-puzzle's graph

Cycles in the graph mean that the later nodes at the bottom could loop back to a node at the top. Our solving algorithm could get “stuck” following this loop and never explore the branch that has the actual solution. In practice, this infinite loop would result in a stack overflow.

We can still use recursion to solve a 15-puzzle. We just need to add our own base case for the maximum number of moves in order to avoid causing a stack overflow. Then, when the maximum number of slide moves is reached, the algorithm will begin backtracking to earlier nodes. If the 15-puzzle solver project can't find a solution in every possible combination of 10 slides, it will try again using a maximum of 11 slides. If the puzzle can't be solved in 11 moves, the project tries 12 moves, and so on. This prevents